Microcontroladores
Semana 7

Semestre 2026-2

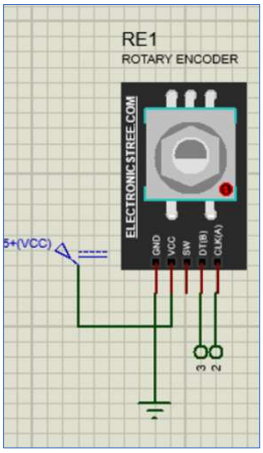
Por Kalun José Lau Gan

Preguntas previas

- ¿Qué es el “BRA $-2”?
  - Es un salto (branch) de dos posiciones de memoria (asumiendo cada posición de un byte) hacia atrás.
- ¿Va a dejar debates en el AV?
  - ¬¬
- La redacción del informe es de manera impersonal
- En I2C. ¿Cómo hacemos con las resistencias de pullup?
  - El PIC18F57Q43 posee pullups especialmente para dicha comunicación, revisar cap 19 del datasheet, de lo contrario se tendrán que colocar de manera externa en los puertos SDA y SCL.

Preguntas previas

- No encuentro el encoder rotatorio en Proteus
  - Si hay:



- ¿Las exposiciones de TP serán en qué horario?
  - Turnos de laboratorio
- ¿Se va formal a la exposición?
  - Si…

Agenda:

- Interrupciones en el PIC18F47Q10 (cap 15 del datasheet)
- Registros implicados en los procesos de interrupción
- Atención de las interrupciones del Timer0
- Interrupciones externas
- Aplicaciones multi-tarea

Interrupciones en un computador

- La PC se maneja en base a Interrupt Requests (IRQs)

IRQ	Usage
0	system timer (cannot be changed)
1	keyboard controller (cannot be changed)
2	cascaded signals from IRQs 8–15
3	second RS-232 serial port (COM2: in Windows)
4	first RS-232 serial port (COM1: in Windows)
5	parallel port 2 and 3 or sound card
6	floppy disk controller
7	first parallel port
8	real-time clock
9	open interrupt
10	open interrupt
11	open interrupt
12	PS/2 mouse
13	math coprocessor
14	primary ATA channel
15	secondary ATA channel

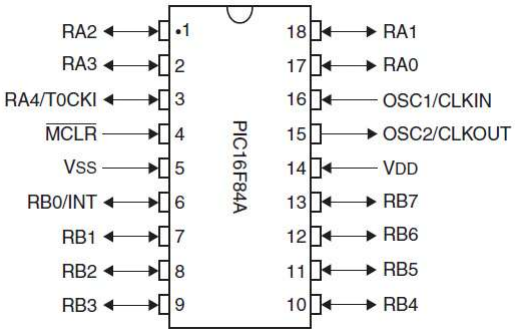


¿Qué son las Interrupciones?

- Es la atención de eventualidades dentro de la ejecución de un programa.
- En general, los microcontroladores poseen diversas fuentes de interrupción pero por defecto esta funcionalidad se encuentra desactivada.
- Es mejor trabajar bajo un esquema de interrupciones que estar con el “polling”
- Se tiene “acción de interrupción” y “fuente ó motivo de la interrupción”
- Por defecto, las CPU atienden solo un evento de interrupción a la vez, y es por eso que se tienen que definir las políticas de atención.
  - Si es por hardware revisar las prioridades
  - Si es por software definir a quien se le va a atender primero.

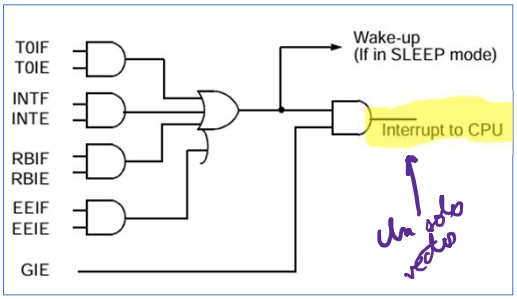
Antes: El PIC16F84A

- Microcontrolador de la gama media de Microchip
- Posee 1K de memoria de programa. 68bytes de memoria de datos.
- Solo un periférico: Timer0



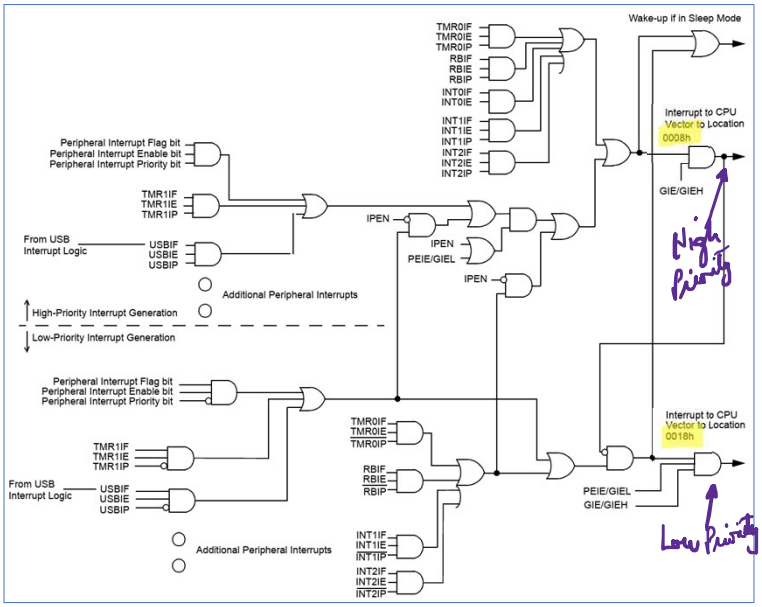
Antes: El PIC16F84A

- Un solo vector de interrupción (canal de atención de las interrupciones por el CPU)
- Cuatro fuentes de interrupción
- En software se tenía que definir la prioridad, o sea, a quien atender primero si es que se tiene mas de una fuente de interrupción habilitada



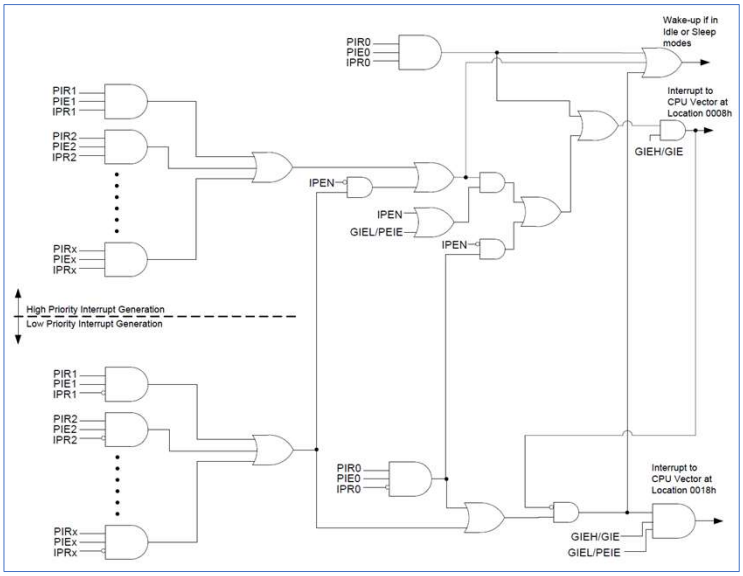
Antes en el PIC18F4550:

- El concepto de prioridades en las interrupciones.
- Se tiene dos prioridades manejadas en hardware: high priority y low priority
- Las políticas de atención:
  - Si llegan ambas a la vez (la alta y la baja), se da prioridad de ejecución a la alta prioridad
  - Si ocurre una interrupción de alta prioridad y luego mientras se esta ejecutando ésta y llega una interrupción de baja prioridad, se sigue ejecutando la alta prioridad y se deja en espera la baja prioridad.
  - Si ocurre una interrupción de baja prioridad y mientras se esta ejecutando ésta y llega una interrupción de alta prioridad, la interrupción de baja prioridad que se encuentra ejecutando pasará a pausarse y de inmediato se atiende la alta prioridad, luego de ejecutarse la de alta prioridad se resume la ejecución de la interrupción de baja prioridad.



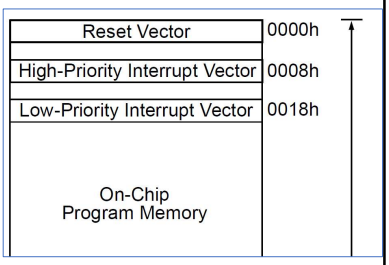
Las interrupciones en el PIC18F47Q10

- Estructura muy similar al PIC18F4550
- Sistema basado en dos fuentes de interrupción: High priority y Low priority



Interrupciones:

- Las interrupciones son **eventualidades** que detienen el flujo normal de operación del microcontrolador.
- Todos los periféricos internos del microcontrolador (Timers, INTs externas, CCP, EUSART, A/D, comparadores analógicos, etc) pueden emitir interrupciones al CPU.
- Las banderas que indican la fuente de interrupción (xxxIF) deberán de bajarse manualmente una vez activados.
- En el PIC18F47Q10 tenemos dos vectores de interrupción:
  - Alta Prioridad (0x0008)
  - Baja Prioridad (0x0018)
- Las interrupciones vienen desactivadas por defecto (GIE=0 y todos los xxxIE=0).
- Las prioridades están desactivadas por defecto (IPEN=0), si están desactivadas, todas van al 0x0008)
- Al activarse las prioridades (IPEN=1) todas las fuentes estarán en alta (xxxIP=1)




```
        ORG 000000H
        bra configuro

        ORG 000008H
        bra INT_ISR_HP

        ORG 000018H
        bra INT_ISR_LP

        ORG 000020H
configuro:      ;---
                ;---
                ;---
```

Sobre las interrupciones en el PIC18F47Q10

- Tener en cuenta que si no se trabaja con prioridades (IPEN=0), todos los eventos se van a la dirección 000008H de la memoria de programa
- Si es que se activa las prioridades, por defecto todas las fuentes de interrupción están en alta prioridad
- Políticas de atención en prioridades:
  - Cuando un evento de alta prioridad y un evento de baja prioridad ocurren al mismo tiempo, el CPU primero atiende al de alta prioridad dejando en cola la atención del de baja prioridad, luego de atender la de alta prioridad se procede a atender al de baja prioridad.
  - Cuando ocurre un evento de de interrupción de alta prioridad, la atiendes y en ese momento recibes una de baja prioridad, el CPU sigue atendiendo al de alta prioridad y luego de terminar atiende la de baja prioridad.
  - Cuando ocurre un evento de baja prioridad, la atiendes y en ese momento ocurre una de alta prioridad, el CPU pone en pausa la atención del de baja prioridad y se va a atender al de alta prioridad, una vez terminado de atender la alta prioridad resume la atención del de baja prioridad.
  - Cuando se reciben dos interrupciones de la misma prioridad. El desarrollador tiene que decidir en el programa a quien atender primero.

Acerca de las interrupciones en microcontroladores

- No siempre se necesitará el uso de interrupciones, todo dependerá de la aplicación a desarrollar.
- Se tiene una mejor respuesta del microcontrolador ante algún evento externo.
- En lo que respecta a aplicaciones relacionadas con el tiempo, es muy necesario configurar las interrupciones ya que añaden la precisión.
- Se tienen acciones de interrupción (interrumpir al CPU) y fuentes de interrupción (quien fue).
- Las prioridades en las interrupciones se recomienda usarlas cuando tengas varias de fuentes de interrupción, no solo una.

La interrupción del Timer0

- Dependiendo del modo de trabajo:
  - Modo 8 bit: El evento de interrupción proviene de la comparación en igualdad entre TMR0H (valor de referencia de comparación) y TMR0L (registro de cuenta) - tmr0_match
  - Modo 16 bit: El evento de interrupción proviene del desborde del registro de cuenta (65535 hacia 0) – tmr0_overflow
- Tener en cuenta el valor que se le coloca en POSTSCALER, este dispositivo contará una cantidad de eventos (tmr0_match o tmr0_overflow) antes de activar la bandera TMR0IF y que va desde 1:1 hasta 1:16.
  - Si POSTSCALER = 1:1, cada evento (tmr0_latch ó tmr0_overflow) activará TMR0IF
  - Si POSTSCALER = 1:2, cada dos eventos (tmr0_match ó tmr0_overflow) activará TMR0IF
- Cuando se levante la bandera (TMR0IF), ésta deberá de bajarse manualmente.
- Para habilitar la interrupción del Timer0:
  - Activar TMR0IE, ubicado en el bit5 del registro PIE0
  - Activar GIE, ubicado en el bit7 del registro INTCON
- De usar prioridades en las interrupciones del Timer0 (modo legacy MVECEN=OFF):
  - Revisar el bit 5 (TMR0IP) del registro IPR0, por defecto TMR0IP=1 (alta prioridad)
  - Revisar bits IPEN, GIEH y GIEL del registro INTCON

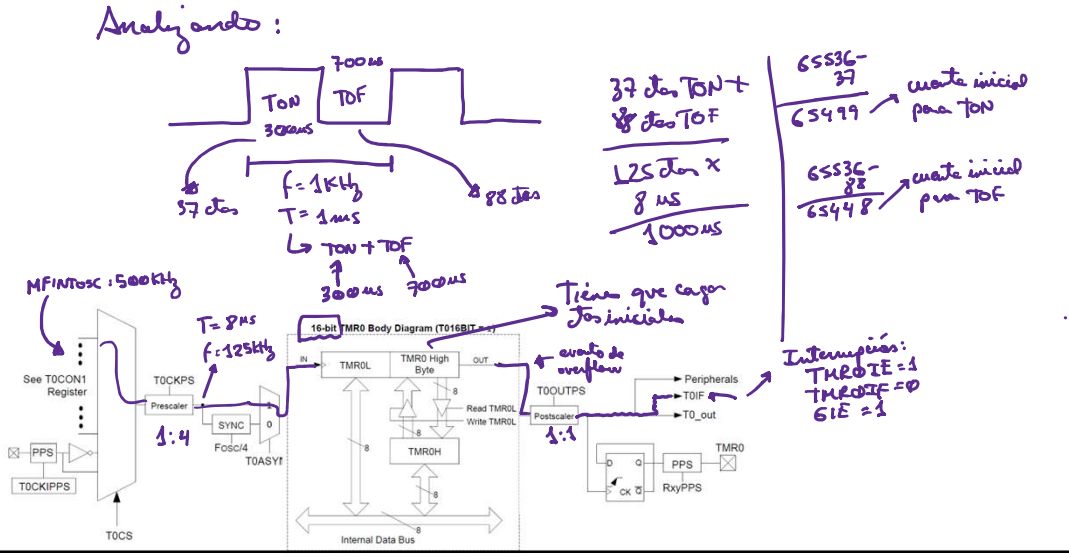
La interrupción del Timer0

- Para el caso del PIC18F47Q10 tenemos lo siguiente:
  - TMR0IP: Definir si Timer0 esta en High o Low priority
  - TMR0IE: Habilitador de la interrución del Timer0.
  - TMR0IF: Bandera de interrupción del Timer0 (se levanta cuando ocurra un evento y se debe de bajar manualmente).
  - GIE/GIEH: Habilitador global de interrupción.
  - PEIE/GIEL: Habilitador de interrupciones de periféricos.
  - IPEN: Habilitador de las prioridades en las interrpciones

Address	Name	Bit Pos.	7	6	5	4	3	2	1	0
0x00 ... 0x0EB4	Reserved									
0x0EB5	IPR0	7:0			TMR0IP	IOCIP		INT2IP	INT1IP	INT0IP
0x0EB6	IPR1	7:0	OSCFIP	CSWIP					ADTIP	ADIP
0x0EB7	IPR2	7:0	HLVDIP	ZCDIP					C2IP	C1IP
0x0EB8	IPR3	7:0	RC2IP	TX2IP	RC1IP	TX1IP	BCL2IP	SSP2IP	BCL1IP	SSP1IP
0x0EB9	IPR4	7:0			TMR6IP	TMR5IP	TMR4IP	TMR3IP	TMR2IP	TMR1IP
0x0EBA	IPR5	7:0	CLC4IP	CLC3IP	CLC2IP	CLC1IP		TMR5GIP	TMR3GIP	TMR1GIP
0x0EBB	IPR6	7:0	CLC8IP	CLC7IP	CLC6IP	CLC5IP			CCP2IP	CCP1IP
0x0EBC	IPR7	7:0	SCANIP	CRCIP	NVMIP					CWG1IP
0x0EBD	PIE0	7:0			TMR0IE	IOCIE		INT2IE	INT1IE	INT0IE
0x0EBE	PIE1	7:0	OSCFIE	CSWIE					ADTIE	ADIE
0x0EBF	PIE2	7:0	HLVDIE	ZCDIE					C2IE	C1IE
0x0EC0	PIE3	7:0	RC2IE	TX2IE	RC1IE	TX1IE	BCL2IE	SSP2IE	BCL1IE	SSP1IE
0x0EC1	PIE4	7:0			TMR6IE	TMR5IE	TMR4IE	TMR3IE	TMR2IE	TMR1IE
0x0EC2	PIE5	7:0	CLC4IE	CLC3IE	CLC2IE	CLC1IE		TMR5GIE	TMR3GIE	TMR1GIE
0x0EC3	PIE6	7:0	CLC8IE	CLC7IE	CLC6IE	CLC5IE			CCP2IE	CCP1IE
0x0EC4	PIE7	7:0	SCANIE	CRCIE	NVMIE					CWG1IE
0x0EC5	PIR0	7:0			TMR0IF	IOCIF		INT2IF	INT1IF	INT0IF
0x0EC6	PIR1	7:0	OSCFIF	CSWIF					ADTIF	ADIF
0x0EC7	PIR2	7:0	HLVDIF	ZCDIF					C2IF	C1IF
0x0EC8	PIR3	7:0	RC2IF	TX2IF	RC1IF	TX1IF	BCL2IF	SSP2IF	BCL1IF	SSP1IF
0x0EC9	PIR4	7:0			TMR6IF	TMR5IF	TMR4IF	TMR3IF	TMR2IF	TMR1IF
0x0ECA	PIR5	7:0	CLC4IF	CLC3IF	CLC2IF	CLC1IF		TMR5GIF	TMR3GIF	TMR1GIF
0x0ECB	PIR6	7:0	CLC8IF	CLC7IF	CLC6IF	CLC5IF			CCP2IF	CCP1IF
0x0ECC	PIR7	7:0	SCANIF	CRCIF	NVMIF					CWG1IF
0x0ECD ... 0x0FF1	Reserved									
0x0FF2	INTCON	7:0	GIE/GIEH	PEIE/GIEL	IPEN			INT2EDG	INT1EDG	INT0EDG

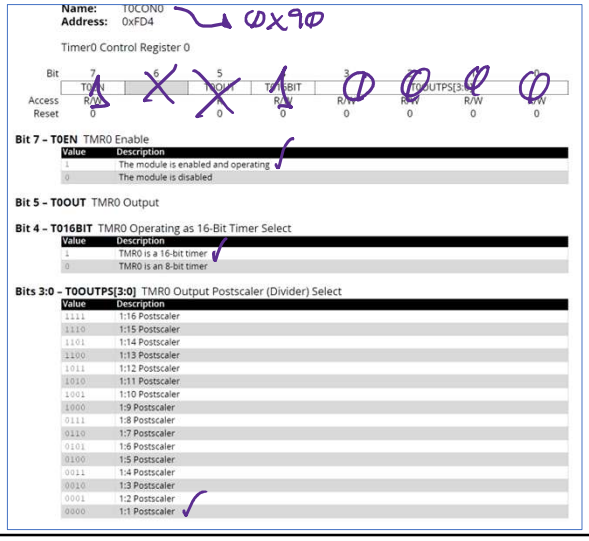
Ejemplo de interrupción del Timer0 en el PIC18F47Q10

- Desarrollar un generador de onda cuadra de 1KHz y 30% de duty cycle con el Timer0 empleando interrupciones



Ejemplo de interrupción del Timer0 en el PIC18F47Q10

- Desarrollar un generador de onda cuadra de 1KHz y 30% de duty cycle con el Timer0 empleando interrupciones



Name: T0CON0 **Address:** 0xFD4

Timer0 Control Register 0

Bit	7	6	5	4	3	2	1	0
	T0EN		T0OUT	T016BIT	T0OUTPS[3:0]			
Access	R/W		R	R/W	R/W	R/W	R/W	R/W
Reset	0		x	0	0	0	0	0

Bit 7 – T0EN TMR0 Enable

Value	Description
1	The module is enabled and operating
0	The module is disabled

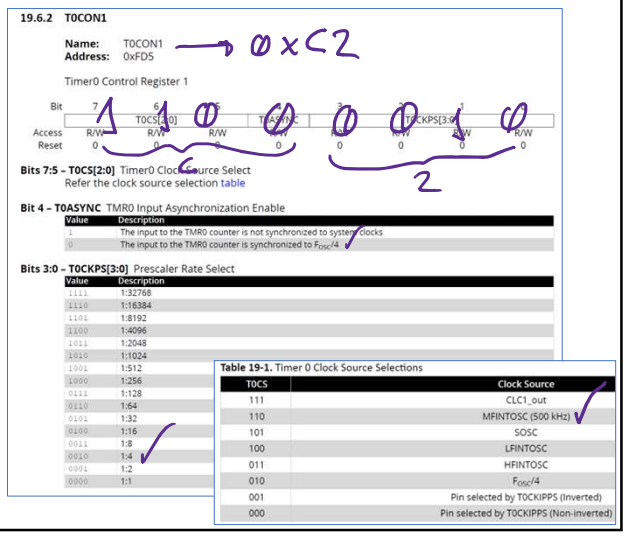
Bit 5 – T0OUT TMR0 Output

Bit 4 – T016BIT TMR0 Operating as 16-Bit Timer Select

Value	Description
1	TMR0 is a 16-bit timer
0	TMR0 is an 8-bit timer

Bits 3:0 – T0OUTPS[3:0] TMR0 Output Postscaler (Divider) Select

Value	Description
1111	1:16 Postscaler
1110	1:15 Postscaler
1101	1:14 Postscaler
1100	1:13 Postscaler
1011	1:12 Postscaler
1010	1:11 Postscaler
1001	1:10 Postscaler
1000	1:9 Postscaler
0111	1:8 Postscaler
0110	1:7 Postscaler
0101	1:6 Postscaler
0100	1:5 Postscaler
0011	1:4 Postscaler
0010	1:3 Postscaler
0001	1:2 Postscaler
0000	1:1 Postscaler



19.6.2 T0CON1

Name: T0CON1 **Address:** 0xFD5

Timer0 Control Register 1

Bit	7	6	5	4	3	2	1	0
	T0CS[2:0]			T0ASYNC	T0CKPS[3:0]			
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 7:5 – T0CS[2:0] Timer0 Clock Source Select
Refer the clock source selection table

Bit 4 – T0ASYNC TMR0 Input Asynchronization Enable

Value	Description
1	The input to the TMR0 counter is not synchronized to system clocks
0	The input to the TMR0 counter is synchronized to $F_{OSC}/4$

Bits 3:0 – T0CKPS[3:0] Prescaler Rate Select

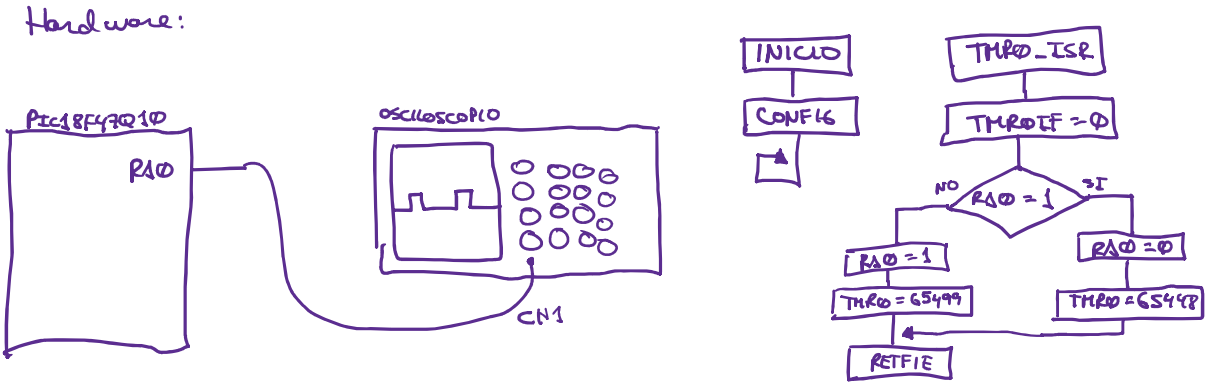
Value	Description
1111	1:32768
1110	1:16384
1101	1:8192
1100	1:4096
1011	1:2048
1010	1:1024
1001	1:512
1000	1:256
0111	1:128
0110	1:64
0101	1:32
0100	1:16
0011	1:8
0010	1:4
0001	1:2
0000	1:1

Table 19-1. Timer 0 Clock Source Selections

T0CS	Clock Source
111	CLC1_out
110	MFINTOSC (500 kHz)
101	SOSC
100	LFINTOSC
011	HFINTOSC
010	$F_{OSC}/4$
001	Pin selected by T0CKIPPS (Inverted)
000	Pin selected by T0CKIPPS (Non-inverted)

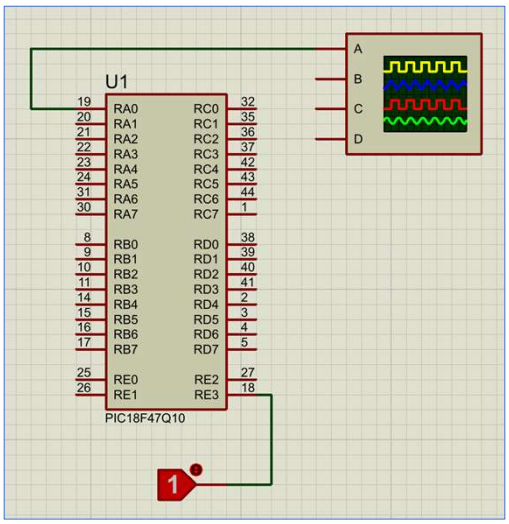
Ejemplo de interrupción del Timer0 en el PIC18F47Q10

- Desarrollar un generador de onda cuadra de 1KHz y 30% de duty cycle con el Timer0 empleando interrupciones



Ejemplo de interrupción del Timer0 en el PIC18F47Q10

- Desarrollar un generador de onda cuadra de 1KHz y 30% de duty cycle con el Timer0 empleando interrupciones



Ejemplo de interrupción del Timer0 en el PIC18F47Q10

- Desarrollar un generador de onda cuadra de 1KHz y 30% de duty cycle con el Timer0 empleando interrupciones

```c
#include <xc.h>
#include "cabecera.h"
#define _XTAL_FREQ 4000000UL

void configuro(void){
    //conf del modulo de oscilador
    OSCCON1 = 0x60;
    OSCFRQ = 0x02;
    OSCEN = 0x40;
    //conf las E/S
    TRISAbits.TRISA0 = 0;
    ANSELAbits.ANSELA0 = 0;
    //conf del Timer0
    T0CON0 = 0x90;
    T0CON1 = 0xC2;
    //conf las INTS
    PIE0bits.TMR0IE = 1;
    PIR0bits.TMR0IF = 0;
    INTCONbits.GIE = 1;
}

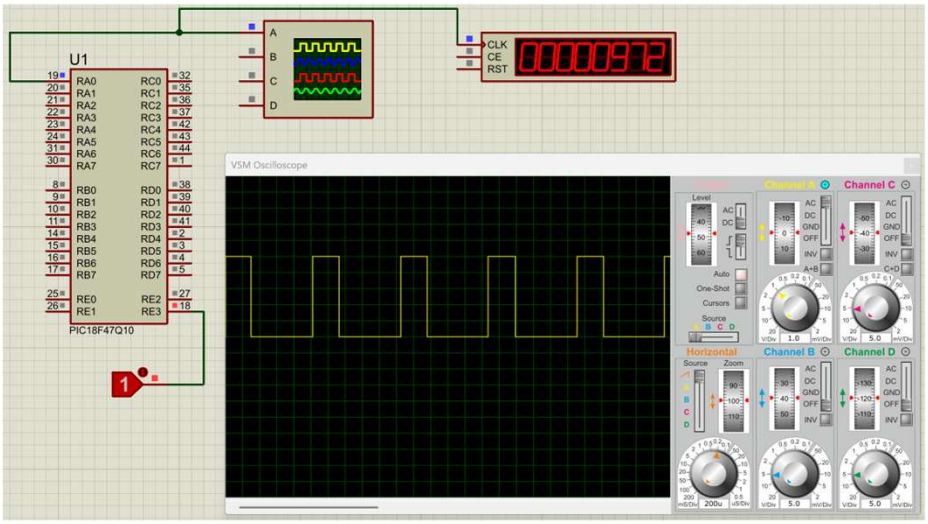
void main(void) {
    configuro();
    while(1){

    }
}
```

```c
void __interrupt() TMR0_ISR(void){
    PIR0bits.TMR0IF = 0;    //bajamos bandera TMR0IF
    if(PORTAbits.RA0 == 1){
        LATAbits.LATA0 = 0;
        //TMR0 = 65448;           //esto no funciona
        TMR0H = 0xFF;
        TMR0L = 0xA8;
    }
    else{
        LATAbits.LATA0 = 1;
        //TMR0 = 65499;           //esto no funciona
        TMR0H = 0xFF;
        TMR0L = 0xDB;
    }
}
```

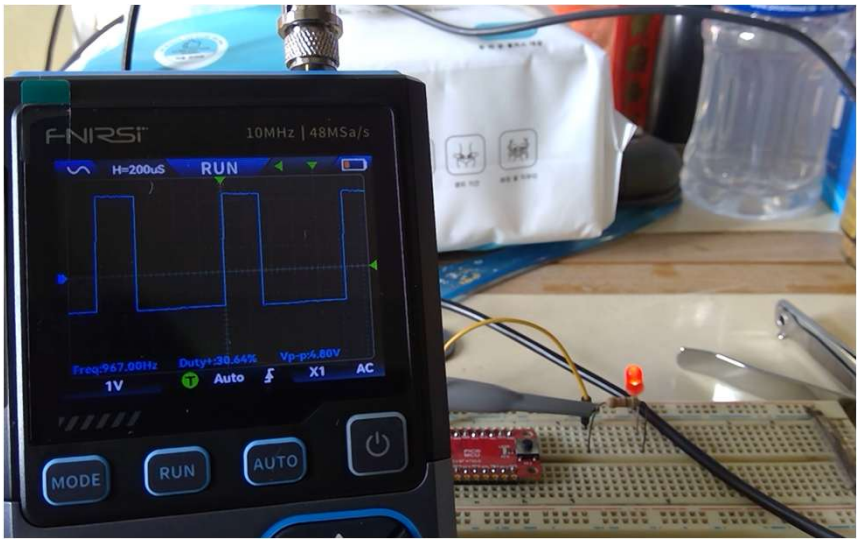
Ejemplo de interrupción del Timer0 en el PIC18F47Q10

- Desarrollar un generador de onda cuadra de 1KHz y 30% de duty cycle con el Timer0 empleando interrupciones



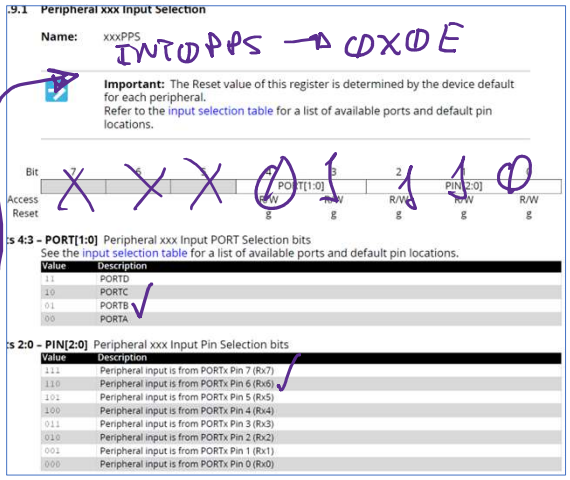
Ejemplo de interrupción del Timer0 en el PIC18F47Q10

- Desarrollar un generador de onda cuadra de 1KHz y 30% de duty cycle con el Timer0 empleando interrupciones



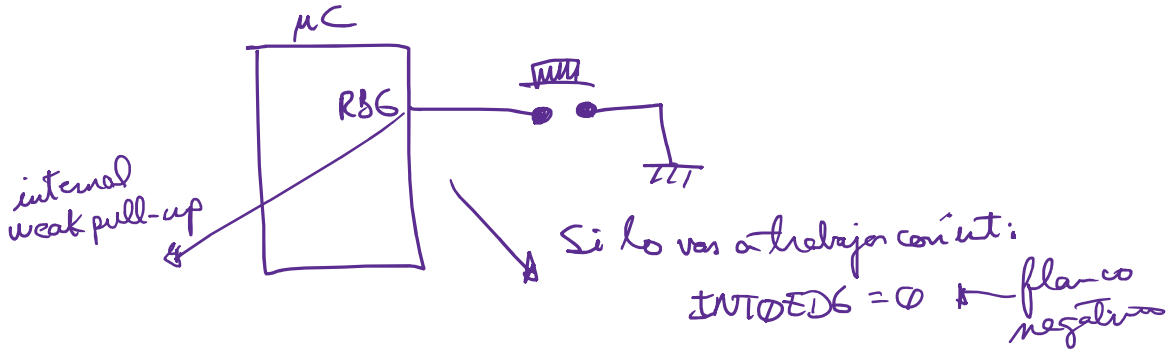
Ejemplo de interrupción externa con el PIC18F47Q10

- Atención de una interrupción que proviene de un pin externo del microcontrolador.
- En el PIC18F47Q10 los puertos A y B son los únicos para poder recibir interrupción externa.
- Se debe de configurar el PPS para asignar el puerto de entrada hacia la interrupción externa INT0, INT1 ó INT2.
  - Por ejemplo para configurar el INT0 con el puerto RB6: INT0PPS



Ejemplo de interrupción externa con el PIC18F47Q10

- Se tiene una configuración para establecer el flanco de detección de la entrada de la interrupción externa: INTxEDG (0 para flanco negativo, 1 para flanco positivo)

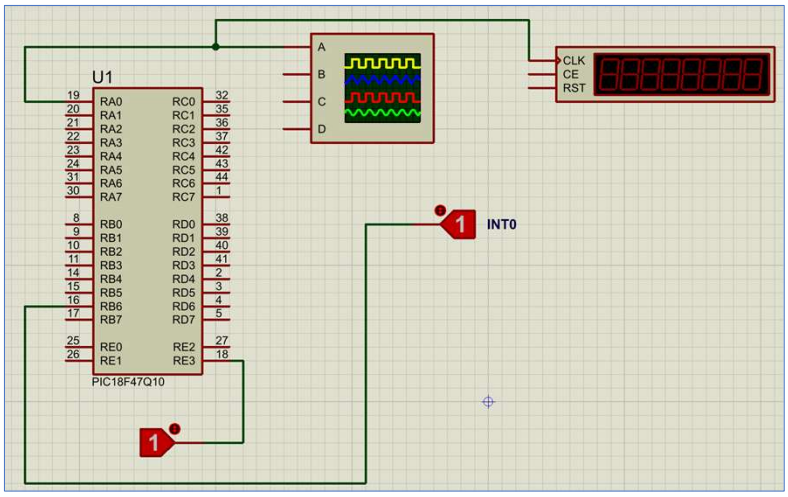


Ejemplo de interrupción externa con el PIC18F47Q10

- No olvidar que la entrada de la interrupción externa debe de configurar como ENTRADA DIGITAL:
  - TRISBbits.TRISB6 = 1; //entrada
  - ANSELBbits.ANSEL6 = 0; //digital
- También no olvidar de aprovechar el uso de la internal weak pullup que tienen los puertos:
  - WPUBbits.WPU6 = 1; //pullup enabled

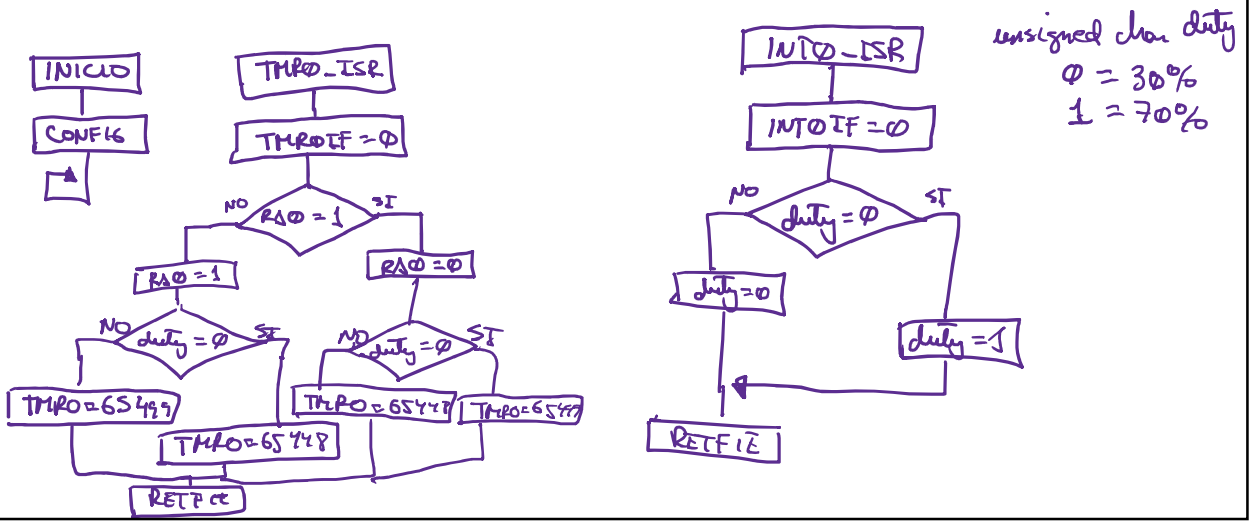
Ejemplo de interrupción externa con el PIC18F47Q10

Añadir la funcionalidad de cambio de duty cycle entre 30% y 70% de la señal generada en el ejemplo anterior

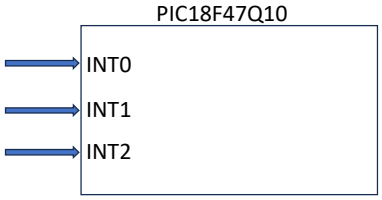


Ejemplo de interrupción externa con el PIC18F47Q10

Añadir la funcionalidad de cambio de duty cycle entre 30% y 70% de la señal generada en el ejemplo anterior



Las interrupciones externas INT0, INT1, INT2

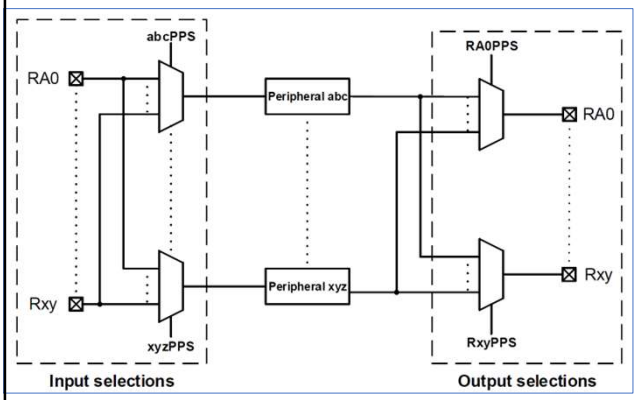


- Interrumpen al CPU del microcontrolador, estas acciones provienen de pines externos (INT0, INT1 e INT2).
- Pines externos (por defecto RB0 para INT0, RB1 para la INT1 y RB2 para INT2). Configurables su asignación por el PPS (registros INTxPPS, revisar cap 18).
- Pueden ser activos en alto (flanco ascendente) o activos en bajo (flanco descendente). Revisar bits INT0EDG, INT1EDG y INT2EDG (registro INTCON).
- Los bits de habilitación de las interrupciones externas son INT0IE, INT1IE e INT2IE. Estos se encuentra en el grupo de registros PIEx
- Las banderas INT0IF, INT1IF e INT2IF deberán de bajarse (acción manual) cuando se activen. Se encuentran en el grupo de registros PIRx.

Las interrupciones externas INT0, INT1, INT2

- Por defecto las tres interrupciones saltan al vector de alta prioridad (000008H) al ser activadas. Si se requiere pasar a baja prioridad (000018H) alguno de ellos se deberá activar la prioridad de las interrupciones (IPEN=1) y configurar los bits INT0IP, INT1IP e INT2IP según requerimientos. Se encuentran en el grupo de registros IPRx.
- Recordar que el GIE/GIEH (habilitador global, global alta prioridad), GIEL (habilitador global baja prioridad) e IPEN (habilitador de prioridades se encuentran en el registro INTCON.

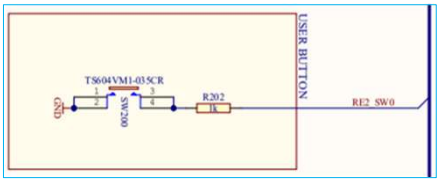
El módulo PPS (Peripheral Pin Select)

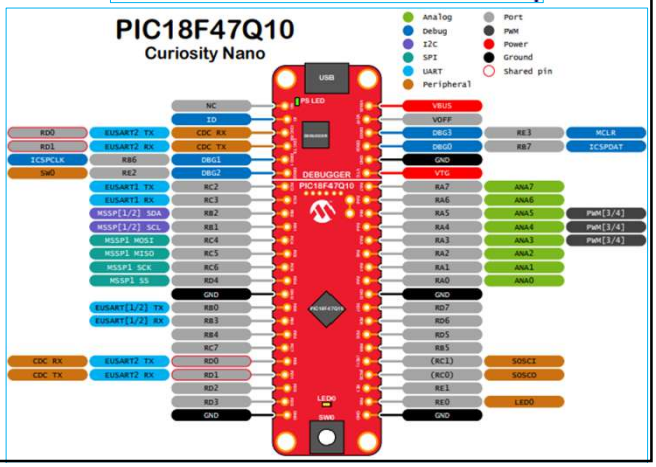


- Referencia cap 21 del datasheet
- Módulo encargado de asignar las señales de los periféricos a los pines del microcontrolador.
- Sólo para señales digitales, tanto de entrada como de salida.
- Con este módulo podrás reasignar los pines de los diferentes periféricos que tiene el microcontrolador, limitado a solo lo que indican las tablas 21-1 (entradas) y 21-2 (salidas).

Ejemplo de PPS aplicado a la INT0







- La INT0 tiene por defecto el RB0 como puerto de entrada
- Tenemos en el Curiosity Nano un botón en RE2 activo en bajo y en drenador abierto.
- ¿Cómo haría para que el RE2 sea la INT0 del microcontrolador?

Ejemplo de PPS aplicado a la INT0

- Respondiendo a la pregunta:
- Revisando la sección de PPS de la hoja técnica:

Table 18-1. PPS Input Selection Register Details

Peripheral	PPS Input Register	Default Pin Selection at POR	Register Reset Value at POR	PORT From Which Input Is Available					
				28-Pin Devices			40-Pin Devices		
Interrupt 0	INT0PPS	RB0	0x08	A	B	—	A	B	—	—
Interrupt 1	INT1PPS	RB1	0x09	A	B	—	A	B	—	—
Interrupt 2	INT2PPS	RB2	0x0A	A	B	—	A	B	—	—
Timer0 Clock	T0CKIPPS	RA4	0x04	A	B	—	A	B	—	—

- Podemos apreciar que para la INT0 solo pueden configurarse pines de los puertos RA y RB, por lo tanto, no es posible usar dicho pulsador integrado del Curiosity Nano para labores de interrupciones externas.

Fin de la sesión